

HLD

Credit91 - High-Level Design

[Credit91 -- Loan Origination & Management System](#)

Next.js 16 + TypeScript + Supabase (Postgres / Auth / Storage) + Vercel

1. Overview

Credit91 is a Loan Origination System (LOS) and Loan Management System (LMS) built as a single Next.js application backed by Supabase. It covers the full lifecycle of an unsecured loan: an applicant submits a loan application, an underwriter reviews and decides on it, and on acceptance a loan is disbursed with a generated repayment schedule that the applicant pays down over time, including an admin-facing collections and interest-accrual workflow for loans that fall behind.

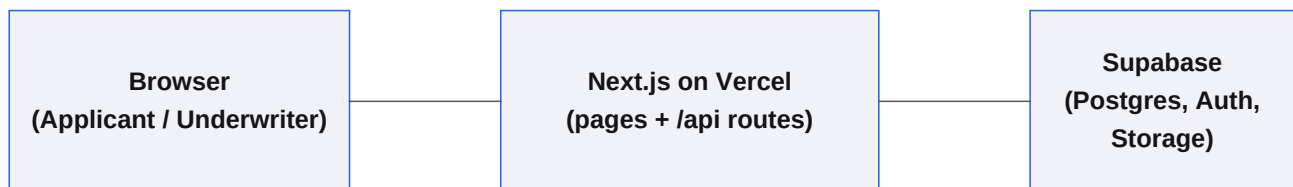
Constraints pitch: Given assessment time and cost constraints, external integrations (credit bureau, payment gateway, notifications) were mocked and the scheduled interest-accrual job was implemented as an admin-triggered action performing identical calculation logic; in production these would be replaced with a real bureau API, payment webhook handler, and a cron-based worker (e.g. Vercel Cron or a queue consumer) with no change to the core domain logic. Email confirmation was disabled to stay within Supabase's free-tier email rate limits; in production this would go through a transactional email provider (e.g. Resend, SES) with confirmation required before first login.

2. Architecture

Stack

Layer	Choice	Rationale
App	Next.js 16 (App Router, TypeScript)	One codebase for pages + API routes; no CORS, no separate backend
DB / Auth / Storage	Supabase (Postgres, Auth, Storage)	One free account replaces 3 separate services
Hosting	Vercel	Deploys the Next.js app directly from GitHub
Interest accrual	Admin-triggered 'Run Accrual' action	Same calculation a scheduled job would run, without cron setup
Credit check / payments	Mocked API routes	No external accounts, keys, or cost

System diagram



The browser talks only to the Next.js app. Server Components and Route Handlers use a Supabase client authenticated with the caller's session cookie, so every query is evaluated under Postgres Row Level Security -- there is no separate authorization layer to keep in sync with the database.

3. External Integrations

Every external, cost-incurring, or account-requiring dependency is isolated behind exactly one function or route, so each has a single, well-defined swap point into a real integration with no ripple effect into the domain model (applications/loans/schedule) or the RLS layer.

Integration	Status	Detail
Credit bureau	Mocked	lib/mockCreditScore.ts returns a pseudo-score biased by income/loan ratio. Swap point: the POST credit-check route body -- same integer score contract, no callers change.
Payment gateway	Mocked	"Simulate Payment" directly marks an installment paid. Swap point: the payments route would initiate a gateway payment intent, and a webhook handler would perform the same DB mutation once the gateway confirms.
Email (auth)	Real	Signup confirmation emails route through a custom SMTP provider (Brevo) connected to Supabase Auth -- a genuine third-party integration, not mocked.
Accrual scheduling	Mocked (manual)	An admin clicks "Run Accrual" instead of a cron job. Swap point: the same run-accrual route body, called by a Vercel Cron / queue worker on a schedule instead of a click.
Document storage	Real	Supabase Storage (S3-compatible), genuinely used with RLS-equivalent bucket policies.
Database / Auth	Real	Supabase Postgres + Auth, genuinely used -- the system of record for every table in this document.

4. Roles & Security Model

There are exactly two roles, stored on a `profiles.role` column populated at signup: `applicant` and `underwriter`. A Postgres trigger (`handle_new_user`) creates the `profiles` row automatically from signup metadata, so the application code never has to synchronize roles by hand.

- Applicants can only see and act on their own applications, documents, credit checks, loans, and payments.
- Underwriters can read every application, run decisions, and view the full audit log and collections list.
- Both the database (Row Level Security policies) and the API routes enforce this -- the API layer checks `profile.role` before allowing an action, and RLS enforces it again at the data layer, so a bug in one layer doesn't expose data.

5. Data Flow

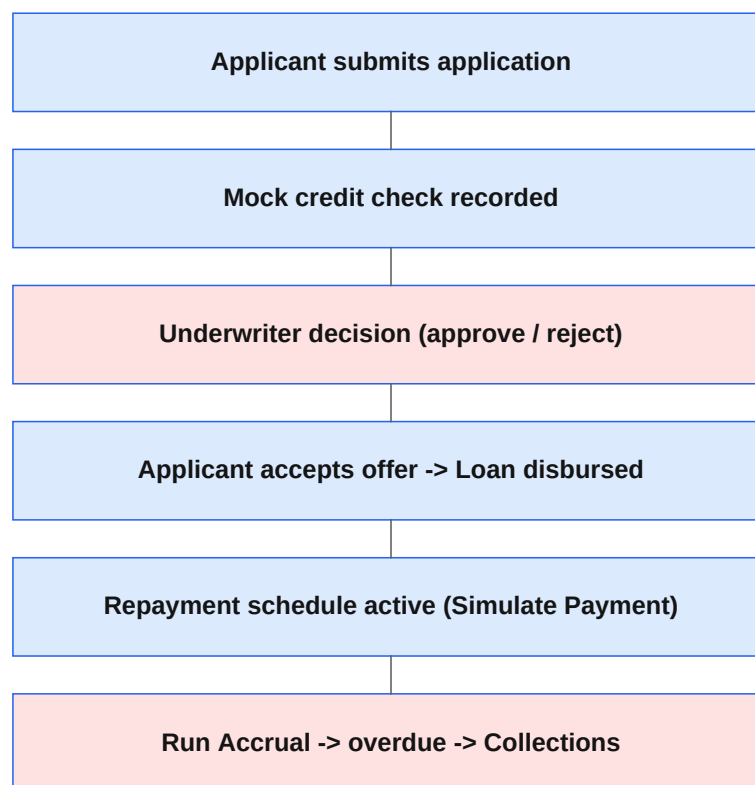
Applicant journey

Sign up (role=applicant) -> submit application (income, employment, amount, tenure) -> upload ID proof document -> trigger mock credit check -> wait for underwriter decision -> if approved, accept offer (creates Loan + amortization schedule, instantly 'disbursed') -> view repayment schedule -> Simulate Payment against the next due installment.

Underwriter journey

Sign up (role=underwriter) -> see all submitted applications with credit score -> open an application -> approve (set amount/rate/tenure) or reject (with reason) -> Collections screen lists loans with installments past their due date -> Run Accrual applies a late-payment penalty and flags the loan delinquent.

End-to-end lifecycle diagram



Rejection (step 3) and delinquency (step 6) are the two terminal/exception branches; every other step is the happy path exercised in the walkthrough video.

6. Design Trade-offs

Five decisions where a real alternative existed, and why the chosen option won for this system's constraints:

1. Single Next.js app vs. separate frontend/backend

Chosen: one Next.js codebase serving both pages and API routes. Alternative: a separate SPA calling an Express/NestJS API. Why: no CORS or cross-service auth-token plumbing, a single deploy target, and the traffic profile doesn't demand independent scaling of UI vs. API. Accepted trade-off: frontend and backend can't be scaled or replaced independently later.

2. Database-level RLS in addition to API-level checks

Chosen: enforce ownership/role checks in both the API route handlers and Postgres Row Level Security policies. Alternative: API-layer checks only. Why: defense in depth -- a missed check in a new route, or a future direct query from a background job or admin script, still can't leak data, because the database itself refuses the query. Accepted trade-off: policies must be maintained in lockstep with schema changes and are debugged in SQL.

3. Manual accrual trigger vs. a real scheduled job

Chosen: an admin-triggered button running the exact accrual calculation. Alternative: Vercel Cron / Supabase Edge Function on a schedule. Why: zero added infrastructure for a time-boxed build, and the calculation logic is identical to what a scheduled job would run -- swapping later is a scheduling change, not a rewrite. Accepted trade-off: unacceptable in production, where collections activity can't depend on someone clicking a button.

4. Fully relational schema vs. a document database

Chosen: Postgres with foreign keys and UNIQUE constraints enforcing one-to-one relationships (one decision, one loan per application). Alternative: a document store keyed by application. Why: the LOS/LMS domain is inherently relational, and constraints let the database itself guarantee integrity instead of application code. Accepted trade-off: schema changes require migrations -- standard, and expected, for financial data.

5. Integer-cents amortization math vs. plain floating point

Chosen: convert to integer cents internally, round each installment, and let the final installment absorb any remainder. Alternative: floating-point arithmetic throughout. Why: float drift compounds over many installments and can leave a loan with a non-zero balance after the final payment -- unacceptable for money. Accepted trade-off: slightly more code, covered by targeted unit tests.

7. Security & Compliance Considerations

- Audit trail: every state-changing action (submit, credit check, decision, upload, disbursement, payment, accrual) writes an `audit_log` row with actor, action, entity, and details -- the foundation a regulatory audit trail is built on.
- Least-privilege data access: RLS enforces isolation server-side, so a compromised or malicious client request still can't read another applicant's data regardless of what the browser sends.
- Secret handling: the Supabase `service_role` key (which bypasses RLS) is only used in the local seed script, never in the deployed app -- the running app always queries as the signed-in user via RLS.
- PII handling: income, employment, and ID documents are access-controlled identically to financial data (RLS on tables, a private Storage bucket, not a public one).
- Explicitly out of scope, not silently ignored: formal KYC/AML verification, PCI-DSS scope (no card data touches this system), and data-retention / right-to-erasure workflows required by regulations such as GDPR or India's DPDP Act.

8. Path to Production

- Replace each mocked integration at its single swap point (see External Integrations).
- Real scheduled job (Vercel Cron or a queue worker) replacing the manual accrual button.
- Route-level rate limiting / abuse protection on public API routes, beyond Supabase Auth's own protections.
- Structured logging and monitoring in place of the current console.error on audit-log failures.
- Schema migrations run through CI (Supabase CLI) instead of a manually-pasted SQL file.
- Postgres connection pooling (Supabase's built-in PgBouncer) sized for serverless function concurrency.
- Confirm currency-specific rounding conventions (e.g. RBI guidance for INR) before relying on this amortization math for real disbursements.

9. Scope Cuts

In scope

- Signup/login with applicant + underwriter roles
- Application form and status tracking
- Document upload (Supabase Storage, one file type)
- Mocked credit score generator
- Underwriter approve/reject with custom rate & tenure
- Loan creation + reducing-balance amortization schedule
- Simulated repayments and balance reduction
- Collections list + manual interest-accrual trigger
- Audit log on every key action
- Row Level Security on every table

Out of scope (deliberate cuts)

- Escrow management
- Real payment gateway / webhook handling
- Real credit bureau integration
- Email/SMS notifications (including signup confirmation email)
- Multi-document KYC / OCR verification
- Fine-grained RBAC beyond the two roles
- A real scheduled cron job for interest accrual

10. Deployment View

The application is a single Vercel project built from the GitHub repository main branch. Two environment variables (NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY) connect it to the Supabase project. The Supabase schema, RLS policies, and storage bucket are provisioned by a single SQL migration file (supabase/migrations/0001_init.sql) that is run once via the Supabase SQL Editor. A seed script (scripts/seed.ts) can populate demo accounts and loans in various states for a live walkthrough.

Live URL: <https://credit91.vercel.app>